

Unit 4: Files

4.1 Files:

- File Naming
- File Structure
- File Types
- File Access
- File Attributes
- File Operations
- Memory-Mapped Files

4.2 Directories:

- Hierarchical Directory System
- Path Names
- Directory Operations

4.3 File System Implementation:

- Implementing Files
- Implementing Directories
- Shared Files
- Disk Space Management
- File System Reliability
- File System Performance

Files

Issues:

- How to store large amount of data into the computer?
- What happens, when process terminates or is killed using some data?
- How to access the same data by multiple processes?
- ***The solution to all these problems is to store information on disks or on other external media called files.***

Files...

- A **file** is a named collection of related information and it normally resides on a secondary storage device such as disk or tape.
- Commonly, files represent programs (both source and object forms) and data; data files may be numeric, alphabetic, alphanumeric, or binary.
- Information stored in files must be persistent, i.e. not be effected by power failures and system reboot.
- The files are managed by OS and the part of the OS that is responsible to manage files is known as the **file system**.

File System Issues

- *How to create files?*
- *How they are named?*
- *How they are structured?*
- *What operation are allowed on files?*
- *How to protect them?*
- *How they are accessed or used?*
- *How to implement?*

File Naming

- Files are an abstraction mechanism.
- When a process creates a file, it gives the file a name. When the process terminates, the file continues to exist and can be accessed by other processes using its name.
- A file is named, *for the convenience of its human users*, and it is referred to by its name.
- A name is a string of characters. The string may be of digits or special characters (e.g. 2, !, % etc).
- Some system differentiate between the upper and lower case character, whereas other system consider the equivalent (like Unix and MS-DOS).

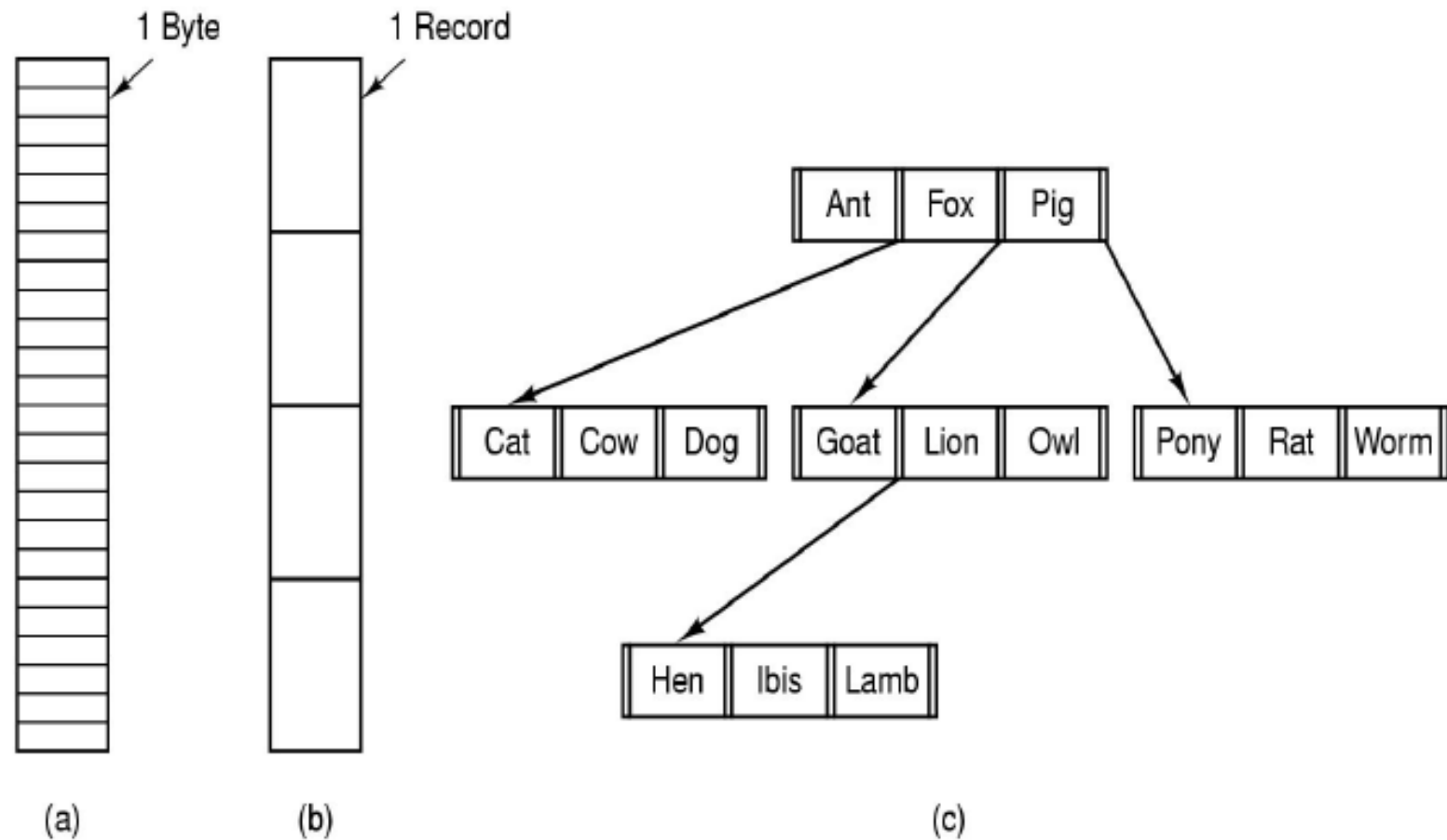
File Naming...

- Normally a string of maximum 8 characters are legal file name (e.g., in DOS), but many recent systems support as long as 255 characters (e.g. Windows 2000).
- Many OSs support two-part file names, separated by period. The part following the period is called the file extension and it usually indicates something about the file (e.g., *file.c* for C source file). But in some systems it may have two or more extensions such as in Unix, a file name like *proc.c.Z* indicates a C source file compressed using *Ziv-Lempel data compression algorithm*.
- In some systems (e.g., Unix), file extensions are just conventions and are not enforced by the OS. However, a C compiler enforces that files, it has to compile, must end in *.c* otherwise it may refuse to compile the program.

File Structure

- Files must have structure that is understood by OS.
- Files can be structured in any of several ways. The most common structures are:
 - Unstructured
 - Record Structured
 - Tree Structured

File Structure



a) Unstructured

b) Record structured

c) Tree structured

File Structure...

Unstructured

- Consists of unstructured sequence of bytes or words.
- OS does not know or care what is in the file. All it sees are bytes. Any meaning must be imposed by user level programs. Both Unix and Windows use this approach.
- Provides maximum flexibility: user programs can put anything they want in their files and name them any way that is convenient.

File Structure...

Record structured

- A file is a sequence of fixed-length records, each with some internal structure.
- Each read operation returns one record, and the write operation overwrites or append one record.
- Many old mainframe systems used this structure however no current general-purpose system uses this structure.

File Structure...

Tree structured

- File consists of a tree of records, not necessarily all the same length, each containing a **key** field in a fixed position in the record.
- The tree is sorted on the key field, to allow rapid searching for a particular key.
- The basic operation is to get the record with a specific key. For the zoo file of above figure, one could ask the system to get the record whose key is *pony*, for example, without worrying about its exact position in the file. Also, new records can be added to the file, with the operating system, and not the user, deciding where to place them.
- Used in large mainframes for commercial data processing.

TU Exam Question 2067/TU Model Question

- Write a short note on ***File Structure***.

File Types

Many OSs supports several types of files. Unix and Windows, for example, have regular files and directories. Unix also has character and block special files.

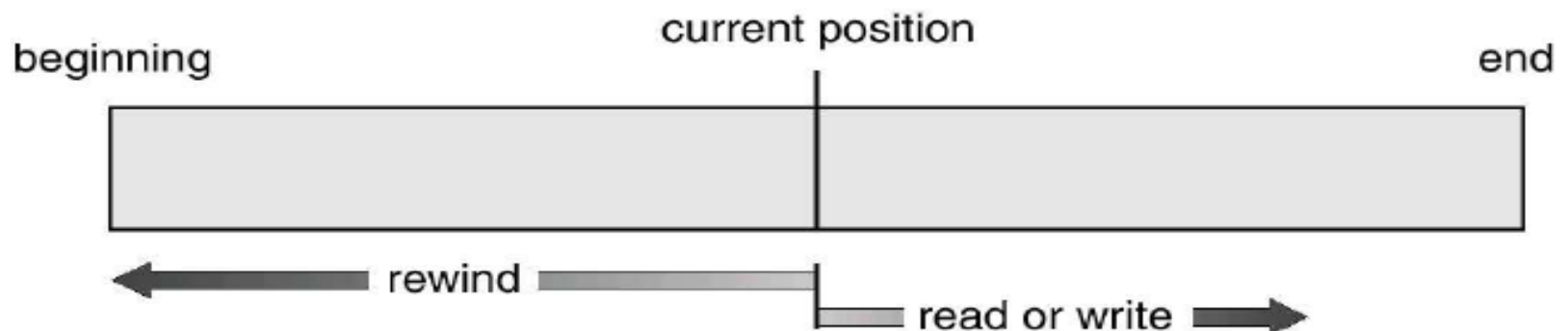
- ***Regular files:*** contain user information. They are generally ASCII files or binary files.
- ***Directories:*** are system files for maintaining the structure of the file system.
- ***Character special files:*** are related to input/output and used to model serial I/O devices such as terminals, printers, and, networks.
- ***Block special files:*** are used to model disks.

File Types...

- ***ASCII files:*** Consists of lines of text. Each line is terminated either by carriage return character or by line feed character or both. They can be displayed and printed as is and can be edited with ordinary text editor.
- ***Binary files:*** Consists of sequence of bytes only. They have some internal structure known to programs that use them (e.g., executable or archive files).
- Many OS use extension to identify the file types; but Unix like OS use a **magic number** to identify file types.

File Access

- Two accessing methods: **Sequential** and **Direct access**.
- **Sequential Access:** It is the simplest access method in which information in the file is processed in order, one record after the other. It is convenient when the storage medium is magnetic tape and was used in many early systems. Rewound is possible.



File Access...

Random Access (Direct/Relative Access):

- Files whose bytes or records can be read out of order are called **random access files**.
- It is based on disk model of file, since disks allow random access to any block.
- Used for immediate access to large amounts of information.
- When a query concerning a particular subject arrives, we compute which block contains the answer, and then read that block directly to provide desired information.
- Operations: **read n** , **write n** (n is block number) and **seek** - to set current position.

File Attributes

- Every file has a name and its data. In addition, all OSs associate other information with each file, for example, the date and time the file was last modified and the file's size. These extra items are called the file's ***attributes***. They are also called ***metadata***.
- The list of file attributes varies from system to system.
- Some common attributes are listed here:

File Attributes

Attribute	Meaning
Protection	Who can access the file and in what way
Password	Password needed to access the file
Creator	ID of the person who created the file
Owner	Current owner
Read-only flag	0 for read/write; 1 for read only
Hidden flag	0 for normal; 1 for do not display in listings
System flag	0 for normal files; 1 for system file
Archive flag	0 for has been backed up; 1 for needs to be backed up
ASCII/binary flag	0 for ASCII file; 1 for binary file
Random access flag	0 for sequential access only; 1 for random access
Temporary flag	0 for normal; 1 for delete file on process exit
Lock flags	0 for unlocked; nonzero for locked
Record length	Number of bytes in a record
Key position	Offset of the key within each record
Key length	Number of bytes in the key field
Creation time	Date and time the file was created
Time of last access	Date and time the file was last accessed
Time of last change	Date and time the file has last changed
Current size	Number of bytes in the file
Maximum size	Number of bytes the file may grow to

Fig: Some possible file attributes

File Operations

Files exist to store information and allow it to be retrieved later. Different systems provide different operations to allow storage and retrieval. Some ***system calls*** provided by OS to perform various operations on files are:

1. **Create:** If disk space is available, it creates a new file without data.
2. **Delete:** Deletes files to free up disk space.
3. **Open:** Before using a file, a process must open it.
4. **Close:** When all accesses are finished, the file should be closed.
5. **Read:** Reads data from file.

File Operations...

- 6. Write:** Writes data to the file.
- 7. Append:** Adds data at the end of the file.
- 8. Seek:** Repositions the file pointer to a specific place in the file.
- 9. Get attributes:** Returns file attributes for processing.
- 10. Set attributes:** To set the user settable attributes when file changed.
- 11. Rename:** Renames a file.

//A simple program to copy a file using system calls

#include <stdlib.h>

#define BUF_SIZE 4096

#define OUTPUT_MODE 700

#define O_RDONLY 400

*int main(int argc, char *argv[])*

{

int in_fd, out_fd, rd_count, wt_count;

char buffer[BUF_SIZE];

if(argc!=3)

exit(1);

in_fd=open(argv[1],O_RDONLY);

if(in_fd<0)

exit(2);

out_fd=creat(argv[2],OUTPUT_MODE);

if(out_fd<0)

exit(3);

while(1)

{

rd_count=read(in_fd,buffer,BUF_SIZE);

if(rd_count<=0)

break;

wt_count=write(out_fd, buffer, rd_count);

if(wt_count<=0)

exit(4);

}

close(in_fd);

close(out_fd);

if(rd_count==0)

exit(0);

else

exit(5);

}

TU Exam Question 2067

- Write a short note on ***File operations***.

TU Model Question

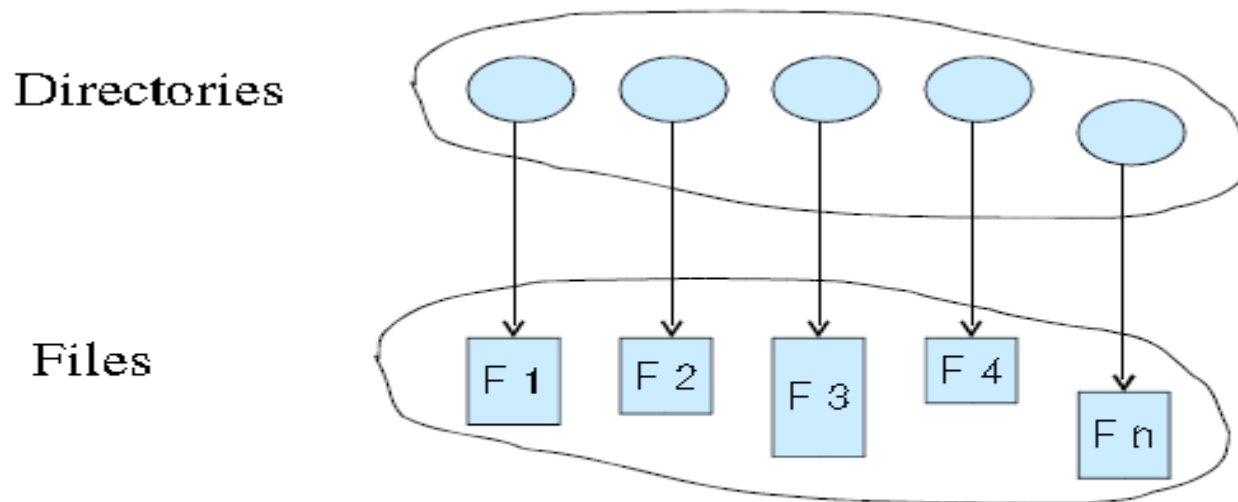
- What is Files? Discuss the most common system calls relating to files.

Memory-Mapped Files

- Performing file operations using standard system calls such as *open()*, *read()*, and *write()* requires a system call overhead and a disk access every time the file is accessed.
- An alternative solution is to use the virtual memory techniques by treating file I/O as routine memory accesses. This approach is known as **memory mapping** a file, allowing a part of a virtual address space to be logically associated with a file.
- Memory mapping a file is possible by mapping a disk block to a page(s) in memory. The initial access to the file proceeds using ordinary demand paging, resulting in a page fault. However, a page sized portion of the file is read from the file system into a physical page. Subsequent reads and writes to the file are handled as routine memory accesses, thereby simplifying file access and usage by allowing file manipulation through memory rather than the overhead of *read()* and *write()* system calls.

Directories

- To keep track of files, file systems have directories or folders, which in many systems are themselves files.
- A **directory** is a node containing information (such as name, location, size, and type) about files.



Directories can have different structures.

Directory Structure

- **Single-Level Directory Systems:** It is the simplest form of directory system having one directory (called the **root directory**) containing all the files.

Advantages: Simplicity and the ability to locate files quickly.

It is often used on simple embedded devices such as telephones, digital cameras, and some portable music players.

Problem: Difficult to manage large amount of files (for example the file names have to be dissimilar) and to manage different user's files.

Directory Structure...

- **Hierarchical Directory Systems:** For modern users with thousands of files, it would be impossible to find anything if all files were in a single directory. Consequently, a way is needed to group related files together. This system creates a hierarchy (i.e. *a tree of directories*). With this approach, there can be as many directories as are needed to group the files in natural ways. This allows multiple users to create their own subdirectories and to organize their files accordingly. To allow to share the directory for different users acyclic-graph-structure is used. Nearly all modern file systems are organized in this manner.

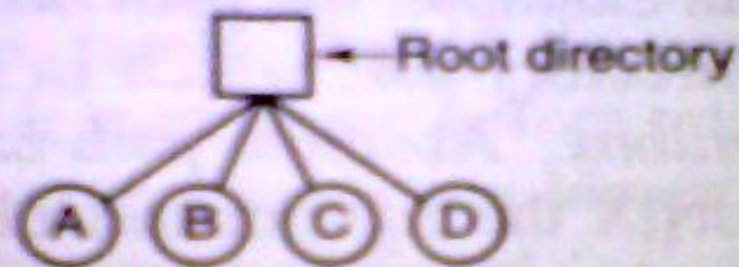


Figure A single-level directory system containing four files.

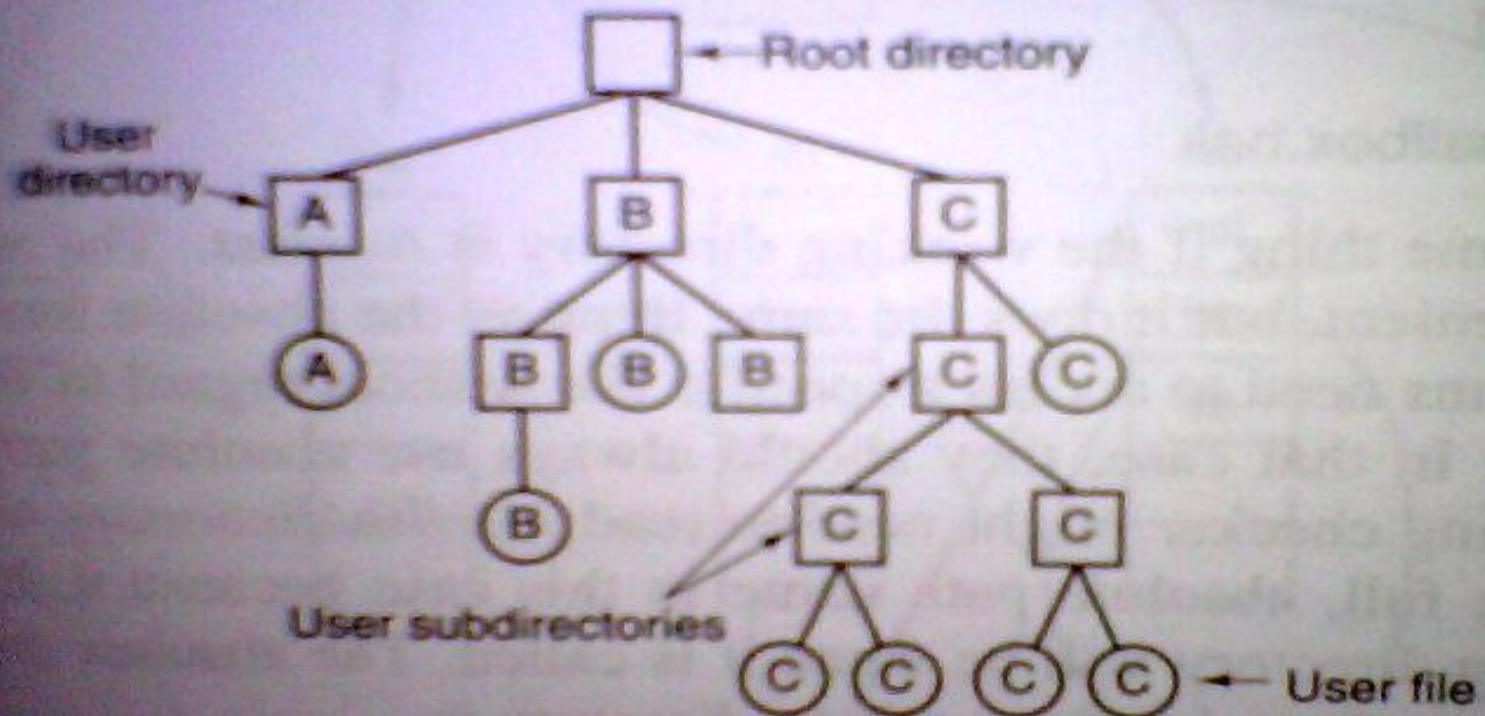


Figure A hierarchical directory system.

TU Exam Question 2066/TU Model Question

- Explain about hierarchical directory systems with diagrammatic examples.

Path Names

- Two different methods are used: ***absolute path name*** and ***relative file name***.
- In the first method, each file is given an ***absolute path name*** consisting of the path from the root directory to the file. For example, the path */usr/ast/mailbox* means that the root directory contains a subdirectory *usr*, which in turn contains a subdirectory *ast*, which contains the file *mailbox*. Absolute path names always start at the root directory and are unique. In UNIX, the components of the path are separated by /. In Windows the separator is \. Thus the same path name would be written as follows in these three systems as:

<i>Windows</i>	<i>\usr\ast\mailbox</i>
<i>UNIX</i>	<i>/usr/ast/mailbox</i>

Path Names...

- The other kind of name is the ***relative path name***. This is used in conjunction with the concept of the ***working directory*** (also called the ***current directory***).
- A user can designate one directory as the current working directory, in which case all path names not beginning at the root directory are taken relative to the working directory. For example, if the current working directory is */usr/ast*, then the file whose absolute path is */usr/ast/mailbox* can be referenced simply as *mailbox*. In other words, the UNIX command

cp /usr/ast/mailbox /usr/ast/mailbox.bak

and the command

cp mailbox mailbox.bak

do exactly the same thing if the working directory is */usr/ast*. The relative form is often more convenient, but it does the same thing as the absolute form.

Directory Operations

The allowed system calls for managing directories are:

- 1. *Create*:** A directory is created. It is empty except for dot and dotdot, which are put there automatically by the system (or in a few cases, by the *mkdir* program).
- 2. *Delete*:** A directory is deleted. Only an empty directory can be deleted. A directory containing only dot and dotdot is considered empty as these cannot usually be deleted.
- 3. *Opendir*:** Directories can be read. For example, to list all the files in a directory, a listing program opens the directory to read out the names of all the files it contains. Before a directory can be read, it must be opened, analogous to opening and reading a file.
- 4. *Closedir*:** When a directory has been read, it should be closed to free up space.

Directory Operations...

5. **Readdir:** This call returns the next entry in an open directory.
6. **Rename:** Renames the directory.
7. **Link:** Linking is a technique that allows a file to appear in more than one directory. This system call specifies an existing file and a path name, and creates a link from the existing file to the name specified by the path. In this way, the same file may appear in multiple directories. A link of this kind, which increments the counter in the file's i-node (to keep track of the number of directory entries containing the file), is called a **hard link**. A variant on the idea of linking files is the **symbolic link** or **soft link**. Instead of having two names point to the same internal data structure representing a file, a name can be created that points to a tiny file naming another file. When the first file is used, for example, opened, the file system follows the path and finds the name at the end. Then it starts the lookup process all over using the new name. Symbolic links have the advantage that they can cross disk boundaries however hard links can be created in one partition only.
8. **Unlink:** A directory entry is removed. If the file being unlinked is only present in one directory (the normal case), it is removed from the file system. If it is present in multiple directories, only the path name specified is removed. The others remain.

Creating Hard/Soft Link using Terminal in Linux

Creating Hard Link:

- *Create a file – \$ cat>file1
... ..(Enter)
Ctrl+d*
- *\$ ls -l (Watch link count – file1 link count = 1)*
- *\$ ln file1 file2 (Create hard link)*
- *\$ ls -l (Watch link count again – file1 link count = file2 link count =2)*
- *\$ ls -i (To see the i-node numbers and find that file1 and file2 have same i-node numbers. OS uses the i-node numbers to recognize files.)*
- *Now if we modify say file2, then file1 will also be modified (Two pointers referencing same address). Note that hard link can be created in one partition only because i-node numbers are partition dependent. The physical file will be only one not depending upon the number of hard links. The content of the file remains as far as one hard link is there so it does not matter if you delete file1 or file2 but not both.*

Creating Hard/Soft Link using Terminal in Linux...

Creating Symbolic/Soft Link: stores path of other file and OS accesses this path

- *Create a file – \$ cat>file1
... ..(Enter)
Ctrl+d*
- *\$ ln -s file1 file2 (Create hard link)*
- *\$ ls -l (Watch soft link character **lrwxrwxrwx**)*
- *\$ rm file1 (To remove file1)*
- *\$ cat file2 (No such file or directory)*
- *\$ ls -l (Path is deleted, watch the **red** highlight)*
- *However you can again create the file file1 so that the soft link is once again activated. Note that the size of soft link file is equal to the count of the number of characters in the file name to which it points.*

File System Implementation

- User's view of the file system is ***file system interface***. Users are concerned with how files are named, what operations are allowed on them, what the directory tree looks like, and similar interface issues.
- Implementor's view of the file system is ***file system implementation***. Implementors are interested in how files and directories are stored, how disk space is managed, and how to make everything work efficiently and reliably. We focus on these issues here.

TU Exam Question 2067

- What do you mean by file system? What are the major difference between file system interface and file system implementation? Explain.

File System Layout

- File systems are stored on disks. Disks can be divided up into one or more partitions, with independent file systems on each partition. Sector 0 of the disk is called the **MBR (Master Boot Record)** and is used to boot the computer. The end of the MBR contains the partition table. This table gives the starting and ending addresses of each partition. One of the partitions in the table is marked as active. When the computer is booted, the BIOS reads in and executes the MBR. The first thing the MBR program does is locate the active partition, read in its first block, called the **boot block**, and execute it. The program in the boot block loads the operating system contained in that partition. For uniformity, every partition starts with a boot block, even if it does not contain a bootable operating system. Besides, it might contain one in the future, so reserving a boot block is a good idea anyway.
- Other than starting with a boot block, the layout of a disk partition varies strongly from file system to file system. Often the file system will contain some of the items shown in figure below. The first one is the **superblock**. It contains all the key parameters about the file system and is read into memory when the computer is booted or the file system is first touched. Typical information in the superblock includes a magic number to identify the file system type, the number of blocks in the file system, and other key administrative information.
- Next might come information about free blocks in the file system, for example in the form of a bitmap or a list of pointers. This might be followed by the i-nodes, an array of data structures, one per file, telling all about the file. After that might come the root directory, which contains the top of the file system tree. Finally, the remainder of the disk typically contains all the other directories and files.

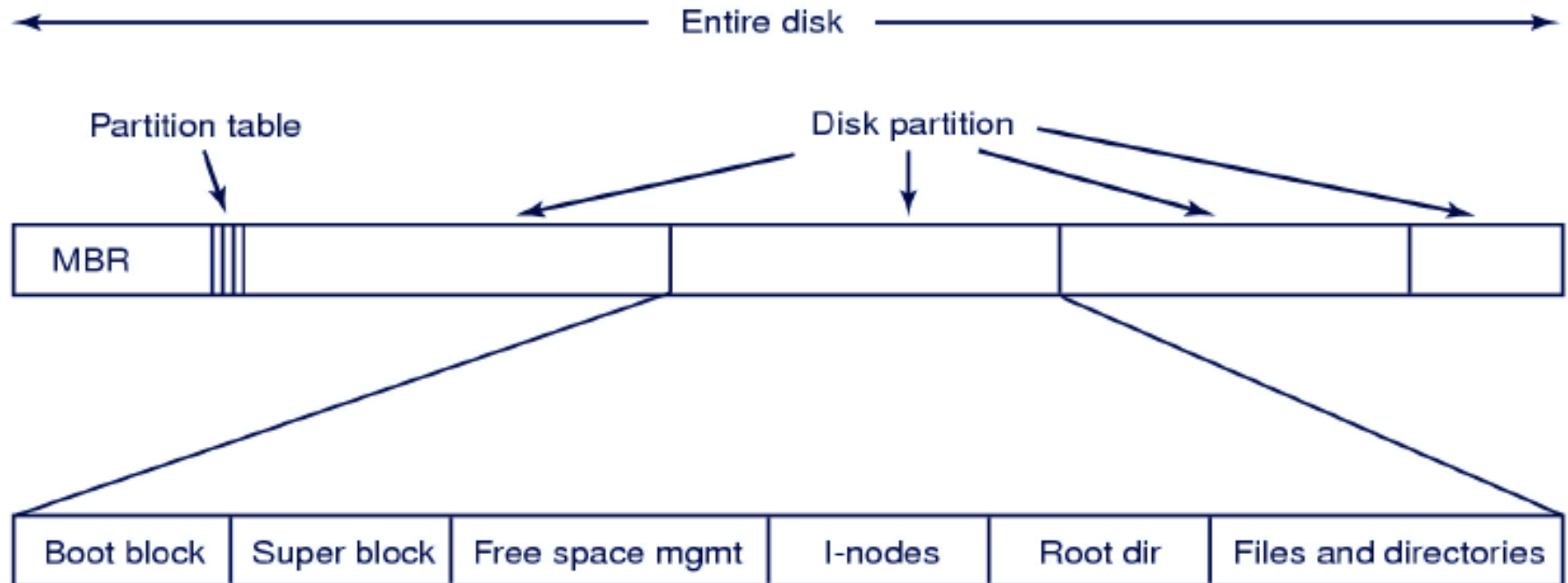


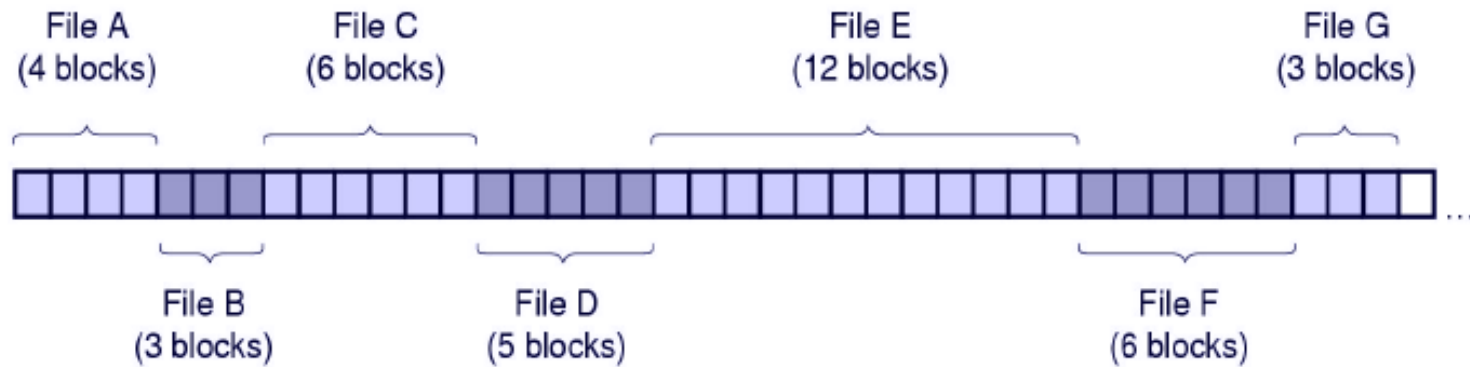
Fig: A possible file system layout

Implementing Files

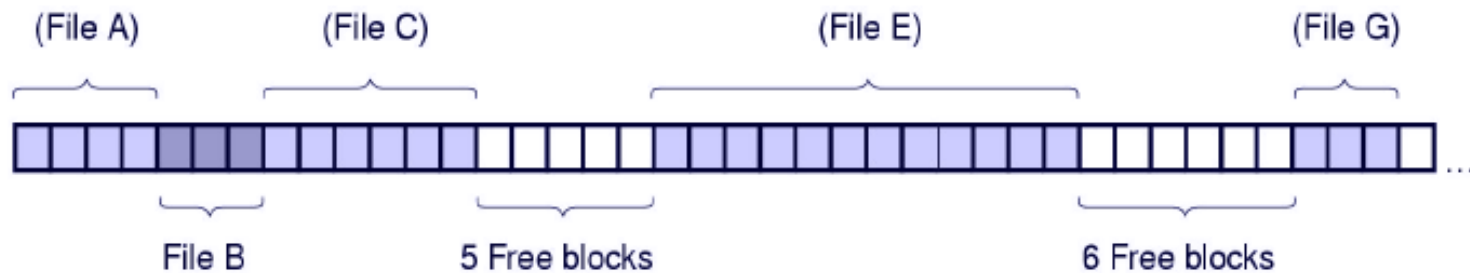
- The most important issue in implementing file storage is keeping track of which disk block go with which file. Various methods exist for file allocation methods:
 - Contiguous Allocation
 - Linked List Allocation
 - Linked List Allocation Using a Table (FAT) in Memory
 - Indexed Allocation (I-nodes)

Contiguous Allocation

- In this allocation scheme, each file occupies a set of contiguous blocks on the disk. With 2-KB blocks, a 50-KB file would be allocated 25 consecutive blocks.
- Disk addresses define a linear ordering on the disk.
- File is defined by the disk address and length in block units.
- Both sequential and direct access can be supported by contiguous allocation.



(a)



(b)

Fig: (a) Contiguous allocation of disk space for seven files.
(b) The state of the disk after files D and F have been removed.

Contiguous Allocation...

- **Advantages:**

- *Simple to implement because keeping track of where a file's blocks are, is reduced to remembering two numbers: the disk address of the first block and the number of blocks in the file. Given the number of the first block, the number of any other block can be found by a simple addition.*
- *High performance because the entire file can be read in a single operation. Only one seek is needed (to the first block). After that, no more seeks or rotational delays are needed, so data comes in at the full bandwidth of the disk.*

- **Problems:**

- *Fragmentation: When files are allocated and deleted the free disk space is broken into holes. Needs compaction which is very expensive.*
- *Dynamic-storage-allocation problem: Searching of right holes. Requires pre-information of file size to place the file in.*

- **Note:** Due to its good performance, contiguous allocation is widely used in CD-ROMs as in this case the file sizes are known in advance and will never change during subsequent use of the CD-ROM file system.

Linked List Allocation

- In this scheme, each file is kept as a linked list of disk blocks. Each block contains pointer to the next block of same file. The disk blocks may be scattered anywhere on the disk.
- Unlike contiguous allocation, every disk block can be used in this method. No space is lost to disk fragmentation (except for internal fragmentation in the last block). Also, it is sufficient for the directory entry to merely store the disk address of the first block. The rest can be found starting from there.

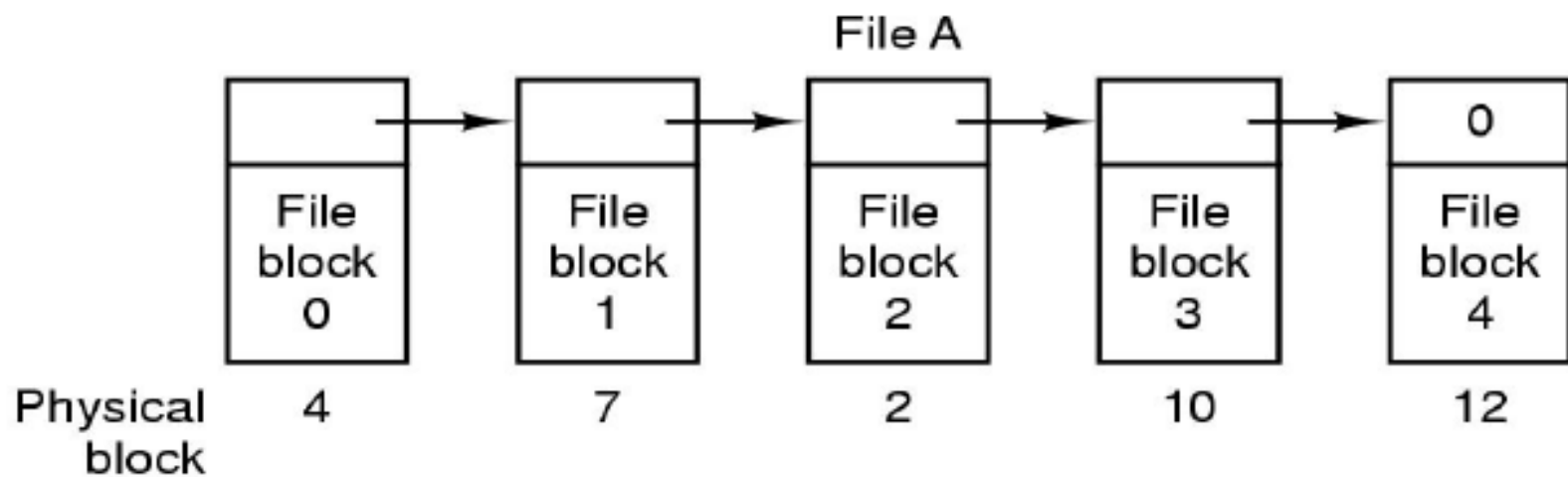


Fig: Storing a file as a linked list of disk blocks

Linked List Allocation...

Problems:

- Although reading a file sequentially is straightforward, random access is extremely slow. To get to block n , the OS has to start at the beginning and read the $n-1$ blocks prior to it, one at a time. Increases seek time.
- The amount of data storage in a block is no longer a power of two because the pointer takes up a few bytes. Since many programs read and write in blocks whose size is a power of two, reads of the full block size requires acquiring and concatenating information from two disk blocks, which generates extra overhead.

Solution: **FAT** (File Allocation Table)

The table has one entry for each disk block containing next block number for the file. This resides at the beginning of each disk partition.

Linked List Allocation Using FAT

- The FAT is used as a linked list. The directory entry contains the block number of the first block of the file. The FAT is looked to find next block until a special *end-of-file* value is reached.

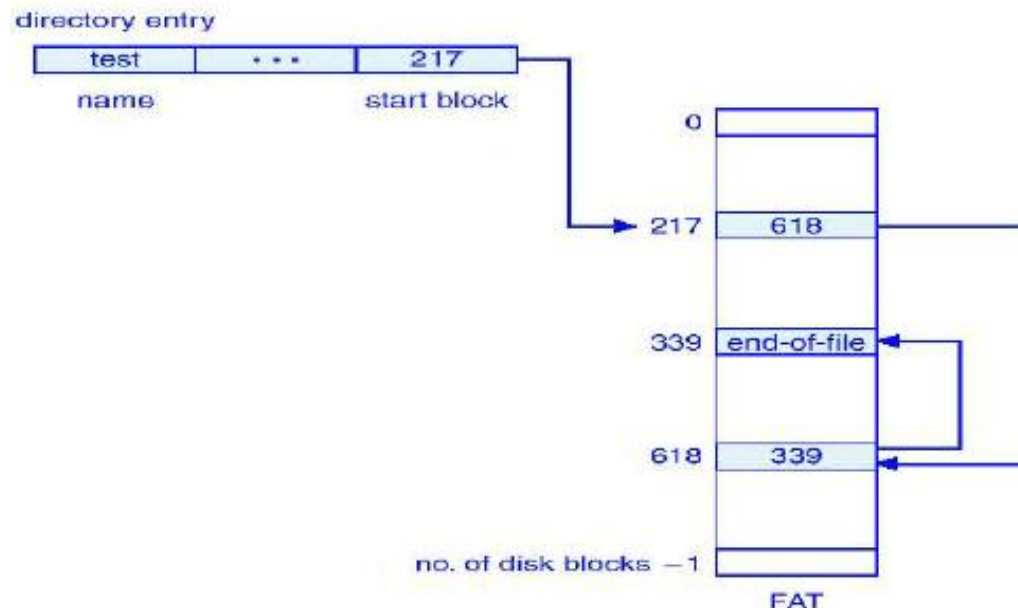


Fig: Linked list allocation using a file allocation table in memory

Linked List Allocation Using FAT

Advantages:

- The entire block is available for data so no overhead like in linked list allocation.
- Random access is improved. Although the chain must be followed to read/write a block, the chain is entirely in memory, so it can be followed without making any disk seeks.

Problem:

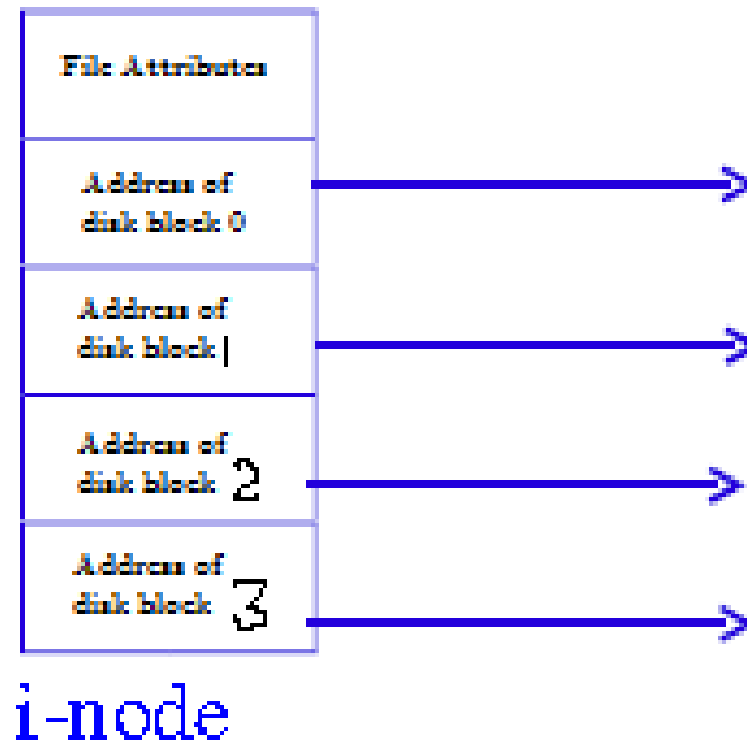
- The entire table must be in memory all the time to make it work. With a 200-GB disk and a 1-KB block size, the table needs 200 million ($200 \times 1024 \times 1024$) entries, one for each 200 million disk blocks. Suppose each entry requires 4 bytes. The table would take up 800MB of main memory all the time. Thus the FAT idea does not scale well to large disks.

TU Exam Question 2066

- Explain how does file allocation table (FAT) manage the files. Mention the merits and demerits of FAT system. A 200 GB disk has 1-KB block size, calculate the size of the file allocation table if each entry of the table has to be 3 bytes.

Indexed Allocation

- To keep track of which blocks belong to which file, each file has a data structure called an **i-node (index-node)**, which lists the attributes and disk addresses of the disk blocks associated with the file. Given the i-node, it is then possible to find all the blocks of the file.
- The big advantage of this scheme over linked files using an in-memory table (FAT) is that the i-node need only be in memory when the corresponding file is open. If each i-node occupies n bytes and a maximum of k files may be opened at once, the total memory occupied by the array holding the i-nodes for the open files is only kn bytes. Only this much space need be reserved in advance. It does not matter whether the disk is 200GB or 2000GB.



Implementing Directories

- Before a file can be read, it must be opened. When a file is opened, the operating system uses the path name supplied by the user to locate the directory entry. ***The directory entry provides the information needed to find the disk blocks of the file.*** Depending on the system, this information may be the disk address of the entire file (contiguous allocation), the number of the first block (both linked list schemes), or the number of the i-node. In all cases, the main function of the directory system is to map the ASCII name of the file onto the information needed to locate the data.
- The file attributes are also stored in the directory (except for systems that use i-nodes in which case, attributes are stored in the i-nodes).
- There are two ways for directory implementation:
 - Linear List
 - Hash Table

Linear List

- The simplest method of implementing a directory is to use a linear list of file names with pointers to the data blocks.
- It requires linear search to find a particular entry.
- **Advantage:** Simple to implement.
- **Problem:** Linear search to find a file.

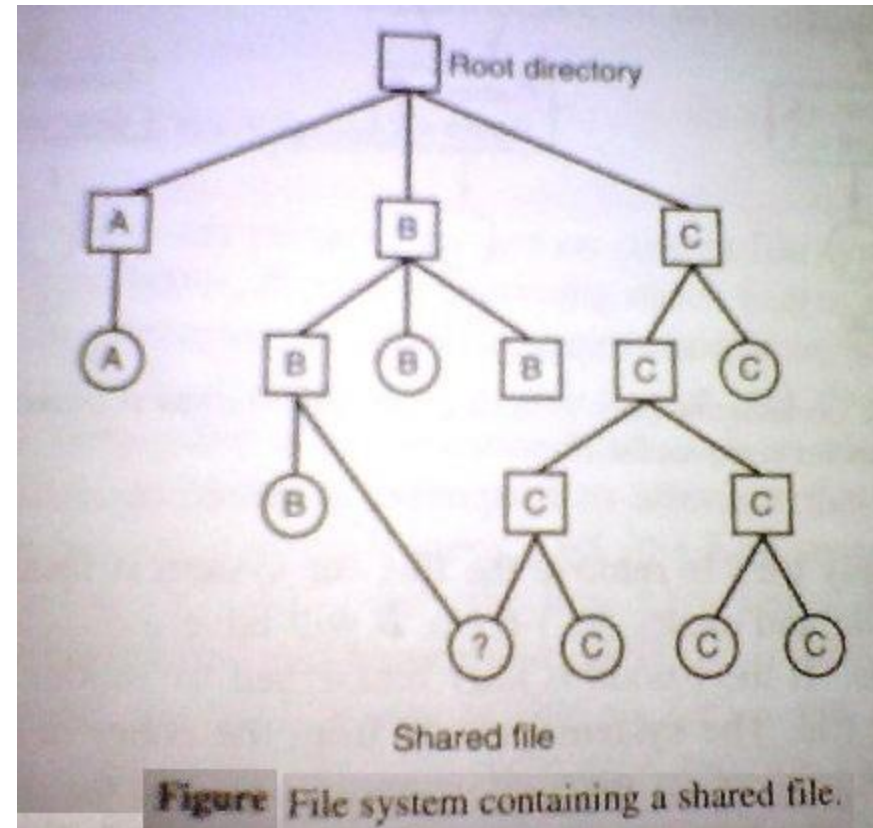
Hash Table

- In this scheme, a linear list stores the directory entries, but a hash table data structure is also used.
- The hash table takes a value computed from the file name and returns a pointer to the file name in the linear list. Therefore, it can greatly decrease the directory search time.
- For resolving hash collisions, each hash entry can be constructed as a linked list.
- **Advantage:** Greatly decreases the file search time.
- **Problem:** Its generally fixed size and the dependence of the hash function on that size.

Shared Files

- When several users are working together on a project, they often need to share files. As a result, it is often convenient for a shared file to appear simultaneously in different directories belonging to different users. Figure on the right shows the hierarchical directory system with one of *C*'s files present in one of *B*'s directories. The connection between *B*'s directory and the shared file is called a **link**. The file system itself becomes a **Directed Acyclic Graph**, or **DAG**, rather than a tree.
- Symbolic linking*** or ***hard linking*** can be used for sharing files.

***Read book (Tanenbaum)
pages 283-285 for details.***



File System Management and Optimization

- Making the file system work is one thing; making it work efficiently and robustly in real life is something quite different.
- ***Issues:***
 - Disk Space Management
 - File System Consistency
 - File System Performance

Disk Space Management

- Nearly all file systems chop files up into fixed-size blocks.
- ***How big the block should be?***
- Using a large block size results in the wastage of disk space.
- Using a small block size increases the seek and rotational delay - low data access rate i.e. low performance.

Refer book (Tanenbaum) pages 292-295 for more information.

Free Space Management

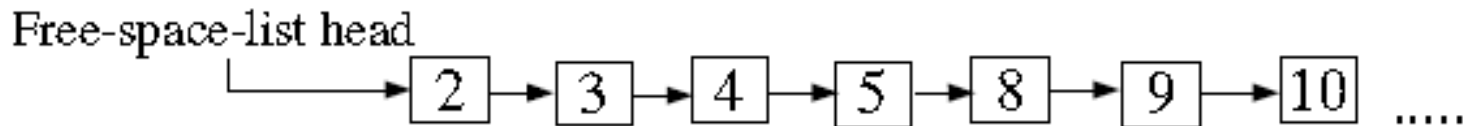
- Once a block size has been chosen, the next issue is how to keep track of free blocks. To keep track of free blocks, the system maintains a ***free-space list***.
- The ***free-space list*** records all free disk blocks - those not allocated to some file or directory.
- To create a file, the system searches the ***free-space list*** for required amount of space, and allocates that space to the new file. This space is then removed from the ***free-space list***.
- When a file is deleted, its disk space is added to the ***free-space list***.
- The ***free-space list*** can be implemented in two ways:
 - Bitmap
 - Linked list

Bit Map or Bit Vector

- Each block is represented by a bit. If the block is free, the bit is 1; if the block is allocated the bit is 0. Thus, a disk with n blocks requires a bit map with n bits.
- For example, consider a disk where blocks 2,3,4,5,8,9,10,11,12,13,17,18,..... are free, and the rest are allocated. The free space bit map would be 0011110011111100011.....
- **Advantage:** Simple and efficient in finding the first free block, or n consecutive free blocks on the disk.
- **Problem:** Inefficient unless the entire bit map is kept in main memory. Keeping in main memory is possible only for small disk. When disk is large the bitmap would be large. For example, a 1.3-GB disk with 512-byte blocks would need a bit map of over 332KB ($1.3 \times 1024 \times 1024 \times 1024 / 512 = 2726297.6$ bits = 340787.2 bytes = 332.8 KB) to keep track of its free blocks.

Linked List

- *Link together all the free disk blocks, keeping a pointer to the first free block in a special location on the disk and caching it in memory.*
- The first block contains a pointer to the next free disk block, and so on.
- The previous example can be represented in linked list as:



- **Advantage:** Only one block is kept in memory.
- **Problem:** Not efficient because to traverse the list, it must read each block (time consuming).

TU Exam Question 2068

How many bits would be needed to store the free-space list under the following conditions if a bitmap were used to implement?

(a) 500,000 blocks total and 200,000 free blocks

(b) 1,000,000 blocks total and 0 free blocks.

Also find how much space is required if it need to be stored in memory.

- **Answer:**

a) 500,000 blocks total and 200,000 free blocks:

Here, Total blocks = 500,000 and Free blocks = 200,000

∴ Number of bits required = 500,000 bits

If 500,000 bits needs to be stored in memory, it requires $(500000/8)$ bytes
= 62500 bytes = $(62500/1024)$ KB = 61.03515625 KB.

(b) Here, Total blocks = 100,000 and Free blocks = 0

∴ Number of bits = 100,000 bits

If 100,000 bits needs to be stored in memory, it requires
 $(100000/8*1024)$ KB = 122.0703125 KB.

File System Consistency

- Many file systems read blocks, modify them and write them out later. If the system crashes before all the modified blocks have been written out, the file system can be left in an inconsistent state.
- To deal with the problem of inconsistent file systems, most computers have a utility program that checks file system consistency. For example, UNIX has *fsck* and Windows has *scandisk*.
- Two kinds of consistency checks are made: blocks and files.
- ***Read pages 304-307 of book Tanenbaum.***

File System Performance

- ***Read pages 307-311 of book Tanenbaum.***